

De Compose

a orquestación.

LA PREGUNTA

¿Y si una sola máquina no basta?

EN UNA FRASE

Empezamos con un archivo que levanta dos contenedores y terminamos con **un grupo de máquinas que se pone de acuerdo**.

Ocho pasos, una decisión.

OCHO PASOS

Contenedor, Compose, su límite, clúster, Pod con Deployment y Service, consenso, Raft, etcd; al final, **la decisión: réplicas o máquina más grande.**

LO QUE TE LLEVAS

- Compose por dentro y dónde se acaba
- Kubernetes básico: Pod, Deployment, Service
- Consenso y Raft desde cero: del problema al algoritmo



EN UNA FRASE Cada paso responde una pregunta, y los ocho desembocan en la misma: **más réplicas o una máquina más grande.**

Lo que quedó de la S4.

DOS ANCLAJES

gRPC es RPC con contrato y deadline; el mismo mensaje pesa **38 bytes en JSON y 9 en Protobuf**.

DE LA S4

gRPC por su nombre RPC con contrato obligatorio y **deadline** en cada llamada

HTTP/2 y HPACK cabeceras comprimidas; HTTP/1.1 las manda completas

Seis piezas y cada servicio, **dueño de su almacén**

Observabilidad

trace id

Caché

copia que se permite estar vieja

Gateway

una sola puerta

Discovery

nombre estable, IP no

El monolito

el sistema de la S04

Config

factor III, fuera de la imagen

Broker

procesar eventualmente



COMO SI FUERA

Un formulario con las casillas numeradas de antemano: si los dos lados conocen el orden, **basta con mandar los valores**.

FarmaYa,

en tres líneas.

FARMAYA

POST /pedidos/{id}/pagar llama a la pasarela (1 a 8 s). **El timeout de 3 s acaba de vencerse.** ¿Qué hace la app?

TRES LÍNEAS

elección A, B o C
porque el requisito que persigo
sacrifico lo que dejo atrás

POST **TIMEOUT DE 3 S VENCIDO**

A · Reintentar de inmediato

at-least-once; exige Idempotency-Key en la pasarela

Sacrifica

latencia extra y depender de que la pasarela deduplique

B · No reintentar

at-most-once; el cliente revisa su estado

Sacrifica

pedidos en limbo y soporte manual

C · Publicar en un broker

responde "en proceso"; consumidor idempotente

Sacrifica

la confirmación inmediata



COMO SI FUERA

Tocas el timbre y nadie abre: no sabes si no sonó o si sonó y ya vienen en camino.
Volver a tocar no sale gratis.

EN UNA FRASE

El timeout dice que no hubo respuesta; **no dice si la operación ocurrió.**

Corre aquí, falla allá.

DEPENDENCIAS DEL ENTORNO

La aplicación depende de **versiones, rutas y librerías de la máquina donde corre.**

LO QUE CAMBIA ENTRE MÁQUINAS

- Java 21 aquí, 17 allá
- Postgres instalada aquí, ausente allá
- La ruta del disco /var/datos no existe allá



EN UNA FRASE El archivo es el mismo en las dos máquinas: **lo que falta es lo que nadie empaquetó junto con él.**

Un proceso con paredes.

CONTENEDOR

Un **proceso aislado** con todos los **archivos que necesita para correr**; comparte el kernel del anfitrión.

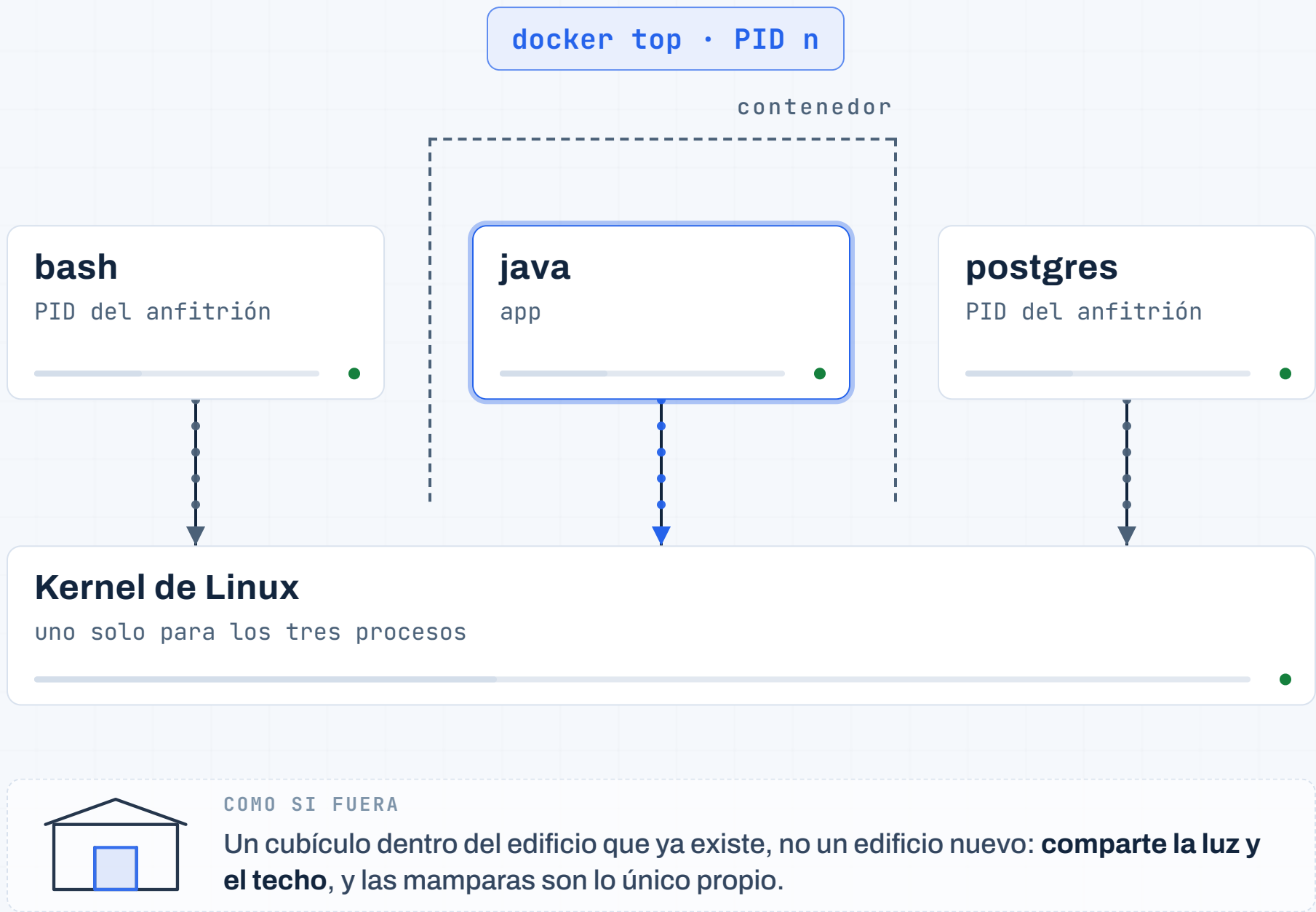
LAS PAREDES, POR DENTRO

Namespaces lo que alcanza a ver: **sus procesos, su red, sus montajes**

Cgroups lo que alcanza a gastar: **CPU y memoria con tope**

Portátil trae sus archivos y corre igual en cualquier anfitrión

EN UNA FRASE **docker exec** no es entrar por SSH: **es otro proceso dentro del mismo cubículo.**



No es una máquina virtual.

MÁQUINA VIRTUAL

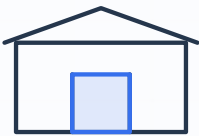
Un **sistema operativo entero con su propio kernel**; un contenedor es un proceso que comparte el kernel.

LA DIFERENCIA

- VM** kernel propio sobre un hipervisor
- Contenedor** kernel compartido, un proceso aislado
- Consecuencia** caben más aplicaciones en menos hardware

MÁQUINA VIRTUAL

- Aplicación**
app.jar
- SO invitado**
kernel propio, drivers, programas
- Hipervisor**
emula el hardware
- Kernel del anfitrión**
el SO de tu máquina
- Hardware**
CPU, memoria, disco



COMO SI FUERA
La máquina virtual construye un anexo nuevo con cimientos propios; el contenedor es un **cubículo dentro del edificio que ya existe.**

CONTENEDOR

- Aplicación en contenedor**
app.jar y sus archivos
- Kernel del anfitrión**
compartido por todos los contenedores
- Hardware**
CPU, memoria, disco

EN UNA FRASE Sin hipervisor ni SO invitado: **el mismo hardware alcanza para más aplicaciones.**

Dos contenedores, un archivo.

DOCKER COMPOSE

Herramienta para definir y correr aplicaciones multicontenedor: un YAML y un comando.

EL EJEMPLO EN COMPOSE

- services: app y db, un contenedor por componente
- docker compose up levanta los dos con un comando
- Sin Compose dos docker run con red, puertos y orden a mano

```
DOCKER-COMPOSE.YML

services:

app:

build: .

ports: "8080:8080"

db:

image: postgres:17-alpine
```

docker compose up --build



COMO SI FUERA

El instructivo que viene con el mueble: dice qué piezas son y en qué orden se arman, así que cualquiera lo arma igual.

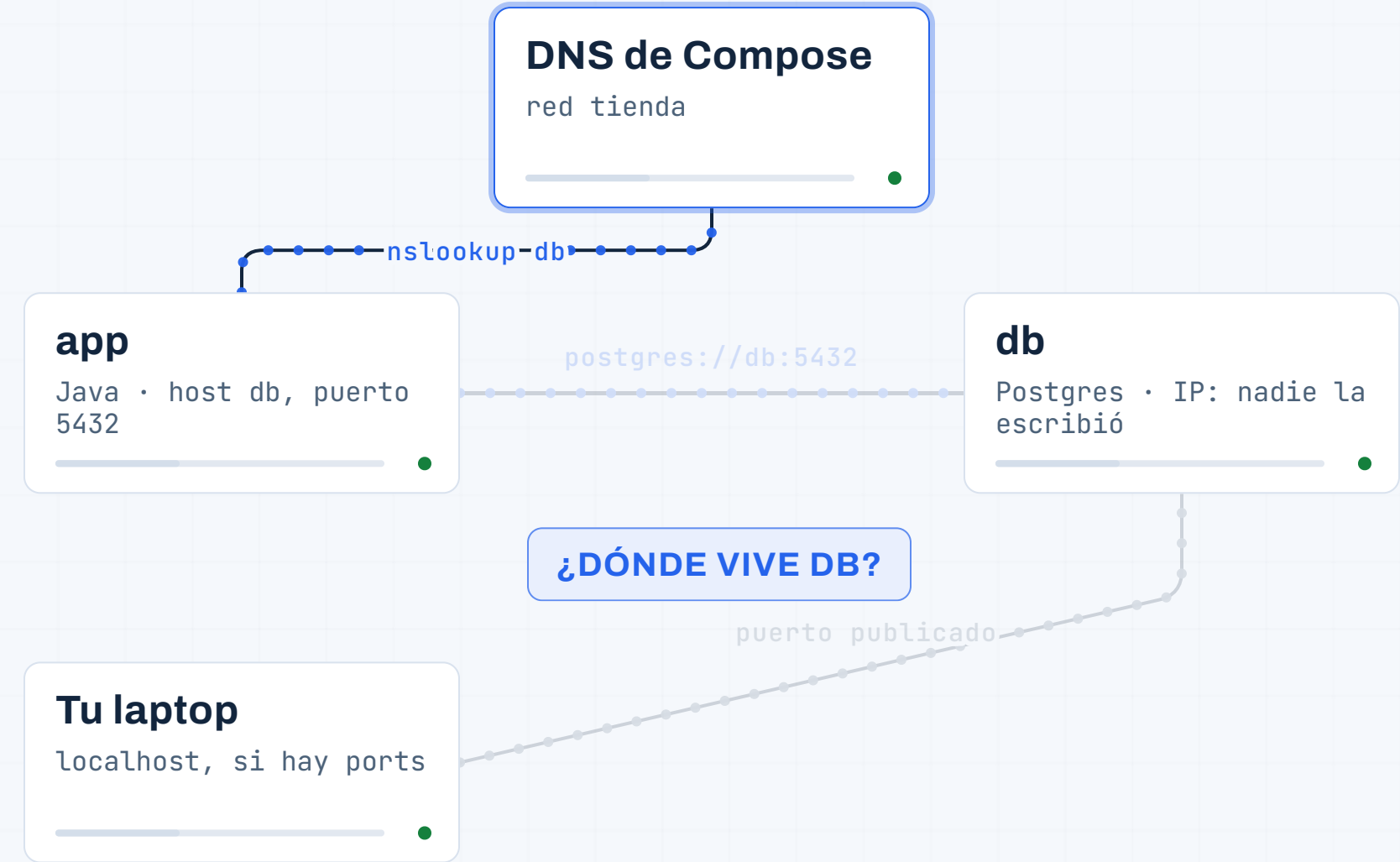
El nombre es la dirección.

RED POR PROYECTO CON DNS

Cada servicio es alcanzable por los demás de su red y **descubrible por el nombre de su servicio**.

EN EL EJEMPLO

- Red tienda, una por proyecto
- Desde app postgres://db:5432, sin IP escrita
- Desde tu laptop solo si publicas el puerto de db



COMO SI FUERA

El conmutador de la oficina: marcas la extensión **db** y él te conecta sin saber en qué escritorio está.

EN UNA FRASE **Por eso nunca hace falta apuntar una IP a mano: el nombre del servicio sobrevive aunque Compose le cambie la IP.**

Arrancó

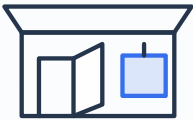
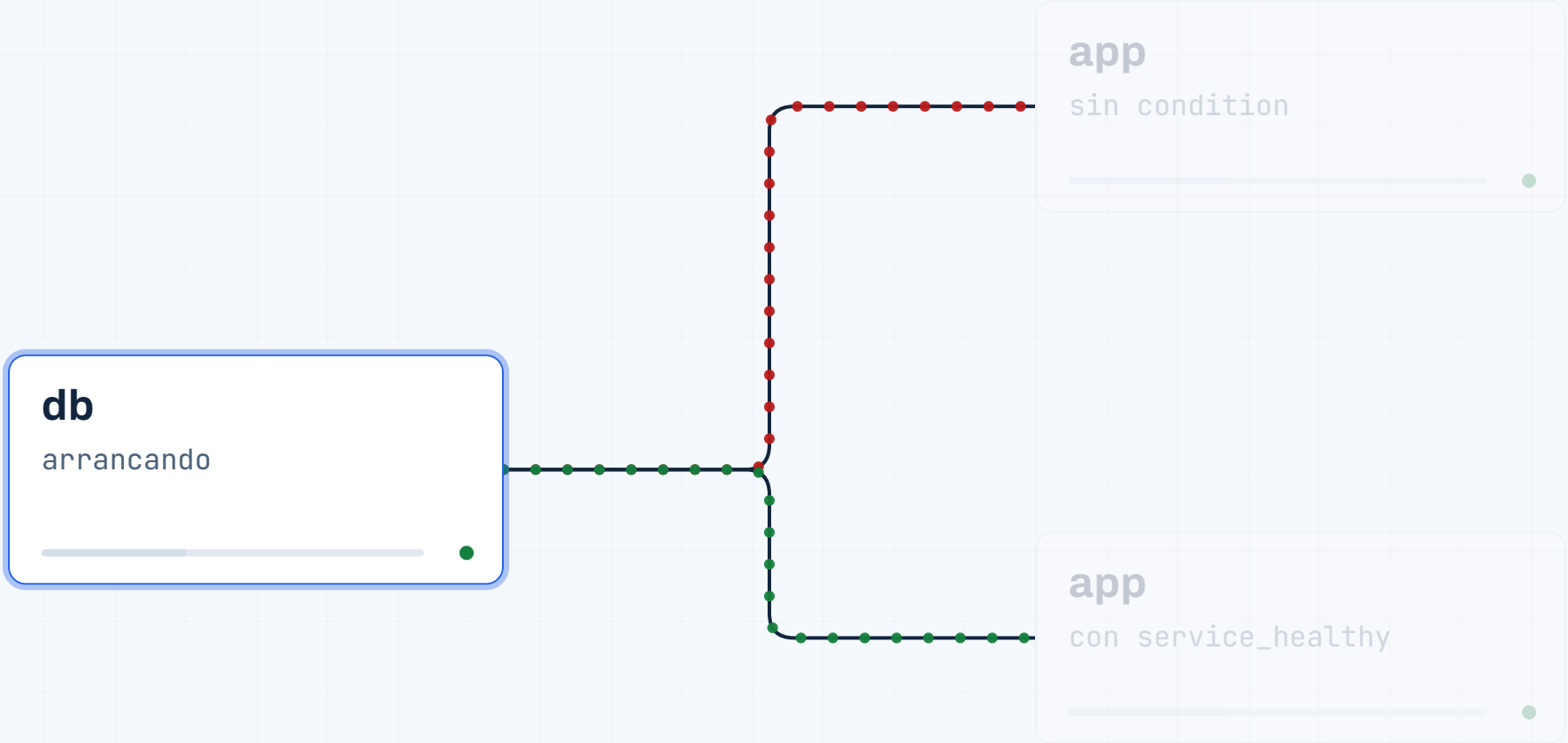
no es listo.

DEPENDS_ON

Ordena el arranque. Solo con **condition: service_healthy** espera además a que el otro responda.

LAS TRES CONDICIONES

- service_started el contenedor existe, nada más
- service_healthy **pasó su healthcheck**
- service_completed_successfully terminó bien



COMO SI FUERA

La puerta abierta del restaurante no es el letrero de **caja abierta**: si pagas antes del letrero, te rechazan.

Tres estados de salud.

HEALTHCHECK

Comando que decide si el contenedor está sano. Tres estados: **starting, healthy, unhealthy**.

EN EL EJEMPLO

db `pg_isready -U tienda cada 5 s`
app `wget /health cada 10 s`
Compose el bloque `healthcheck` entra al `YAML`

db

recién creado · sin respuesta
sana aún

ESTADO **STARTING**

pg_isready

`-U tienda · cada 5 s`

COMPOSE.YAML · DB

`healthcheck:`

`test: pg_isready -U tienda`

`interval: 5s`

`retries: 5`

`docker inspect · Health.Status`



COMO SI FUERA

La revisión médica de rutina: nadie te declara sano por tu aspecto sino **por la prueba que alguien definió**, y mientras no hay resultado sigues en observación.

EN UNA FRASE De los tres estados, el que le importa a `depends_on` es uno solo: **healthy**.

Cuánto tarda en darse cuenta.

POR OMISIÓN · INTERVAL 30S · TIMEOUT 30S · RETRIES 3

FALLO 1 · 30 S

FALLO 2 · 60 S

FALLO 3 · 90 S

VALORES POR OMISIÓN

interval 30s, timeout 30s, retries 3:
tres fallos consecutivos para
unhealthy, del orden de 90 s.

POR OMISIÓN Y EN EL EJEMPLO

Dockerfile interval 30s · timeout 30s · retries 3
También start-period 0s · start-interval 5s
db del ejemplo interval 5s · retries 5

db dejó de responder · cada marca es un intento fallido

ESTADO **UNHEALTHY · 90 S**



COMO SI FUERA

Las rondas del vigilante: si pasa cada media hora, **el problema puede durar media hora escondido**; si pasa cada cinco minutos, no.

EN UNA FRASE Por eso el ejemplo no se queda con los valores por omisión: **quiere enterarse en segundos, no en minuto y medio.**

El gafete

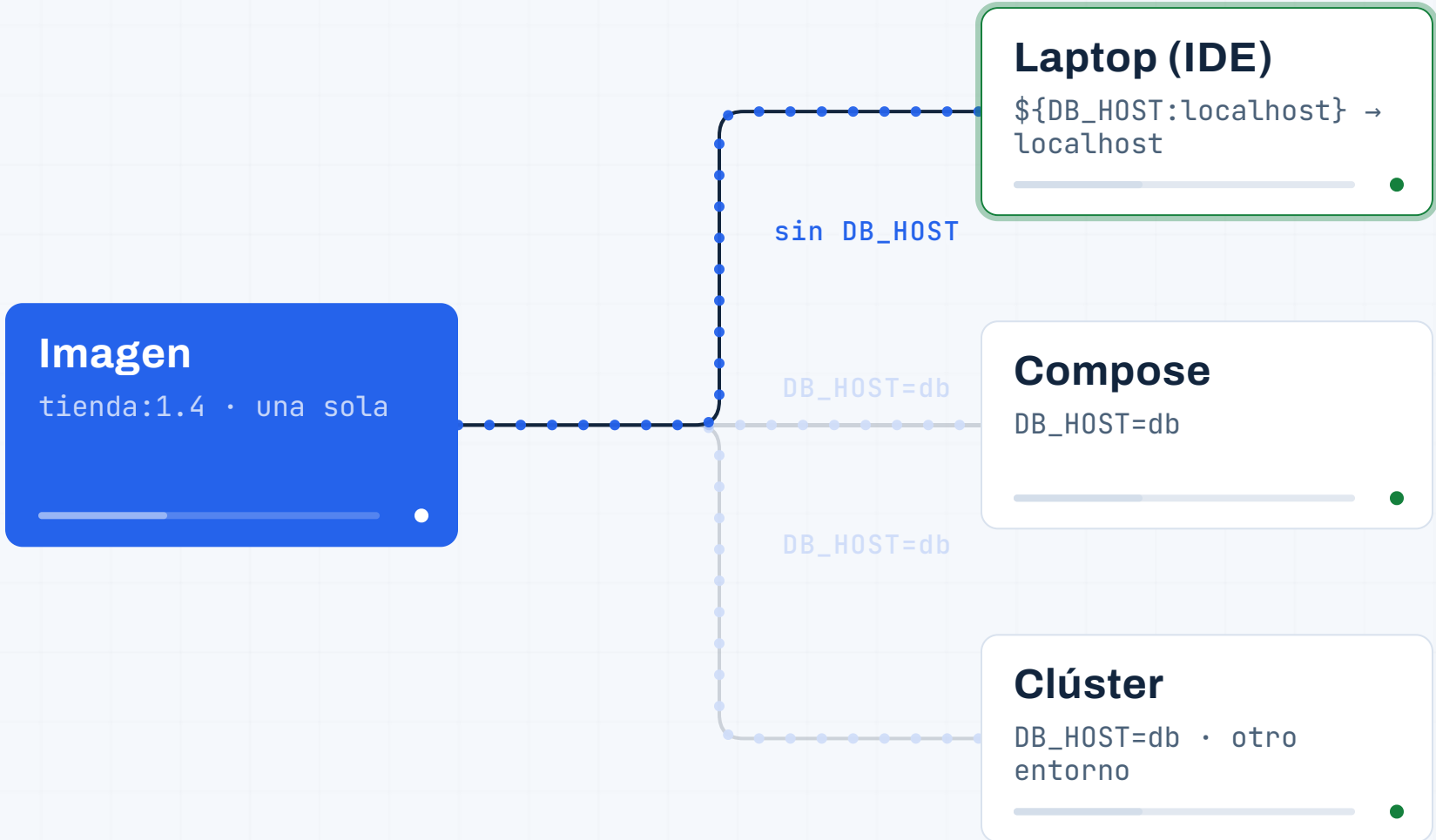
y el uniforme.

VARIABLES DE ENTORNO

La **misma imagen** corre en laptop, Compose y clúster; lo que cambia viaja por el entorno, no en la imagen.

EN EL COMPOSE Y EL IDE

compose environment: DB_HOST: db
application.yml \${DB_HOST:localhost} para el IDE
env_file saca las variables del YAML; environment gana



COMO SI FUERA

El uniforme es igual para todos y no dice nada; **el gafete dice en qué sucursal entras hoy**, y a nadie se le ocurre coserlo al uniforme.

EN UNA FRASE

Por eso nunca va una contraseña dentro de la imagen: **el secreto vive en el entorno, no en el build.**

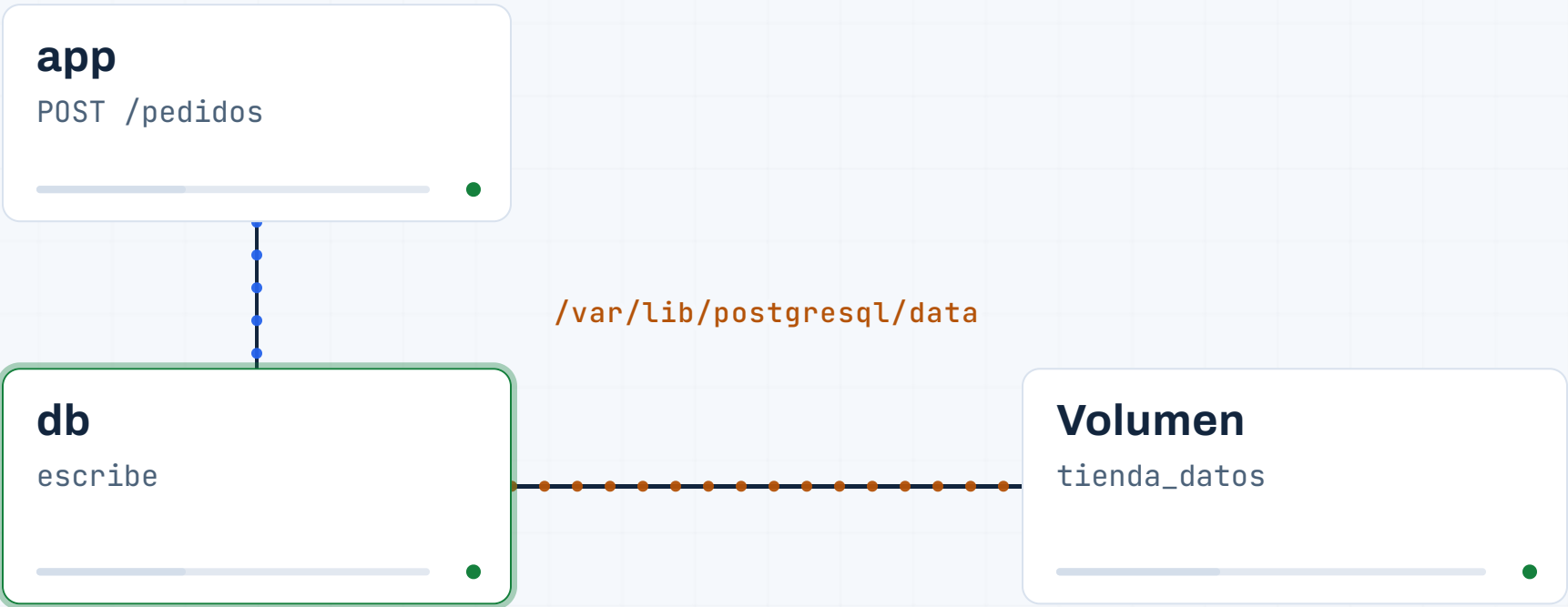
Lo que sobrevive.

VOLUMEN

Existe **fuera del ciclo de vida del contenedor**: al destruirlo muere su capa de escritura, no el volumen.

TRES COMANDOS

docker compose down conserva el volumen
down -v borra los datos
volume ls lo ves ahí aunque no haya contenedor



COMO SI FUERA

El pizarrón se borra al final del día; **la libreta del cajón sigue ahí** con todos los registros.

EN UNA FRASE

El contenedor es desechable; **el volumen es el que recuerda.**

¿Qué garantiza

service_healthy?

COMPROBACIÓN 1

En el compose del ejemplo, app declara depends_on: db con condition: service_healthy. ¿Qué garantiza **exactamente**?

OPCIONES

- A Postgres acepta conexiones
- B el contenedor existe, nada más
- C app y db comparten red
- D si db cae, app se reinicia

COMPOSE.YAML · APP

depends_on:

db:

condition: service_healthy

db

healthcheck: pg_isready

ESTADO **HEALTHY**

app

depends_on: db ·
service_healthy

EN UNA FRASE Las cuatro suenan razonables; la pregunta es **cuál de ellas garantiza service_healthy.**

15 / LÍMITE

Quién lo levanta.

POLÍTICA DE REINICIO

no, on-failure[:n], always o unless-stopped; **aplica solo si el contenedor vivió al menos 10 s.**

EN EL EJEMPLO

restart: unless-stopped en app y en db
kill app vuelve en **5 s**
stop app no vuelve: lo paraste tú

Demonio de Docker

aplica restart cuando el proceso sale

vigila

app

Up · healthy

restart: unless-stopped

SI VIVIÓ AL MENOS 10 S

CUATRO POLÍTICAS

no nunca

on-failure[:n] solo con error

always siempre

unless-stopped salvo que lo pararas tú



COMO SI FUERA

El gerente revisa el contrato antes de reanimar al trabajador desmayado: **unless-stopped** no aplica si tú lo mandaste a casa.

EN UNA FRASE **Kill y stop se ven igual desde afuera: el contenedor muere; la política decide si alguien lo levanta.**

Proceso, demonio y máquina.

TRES FALLAS DISTINTAS

Proceso: lo levanta la política;
demonio: **always y unless-stopped vuelven**; máquina apagada: nadie.

QUÉ SOBREVIVE

Proceso on-failure, always, unless-stopped
Demonio on-failure **no** vuelve
Máquina el demonio se apagó con ella



LA POLÍTICA LO LEVANTA



COMO SI FUERA
Si se cae el edificio, el gerente estaba adentro: **no hay contrato que valga** si quien lo aplica se apagó con la máquina.

EN UNA FRASE **always suena a siempre, pero no sobrevive al apagón de la máquina.**

17 / LÍMITE

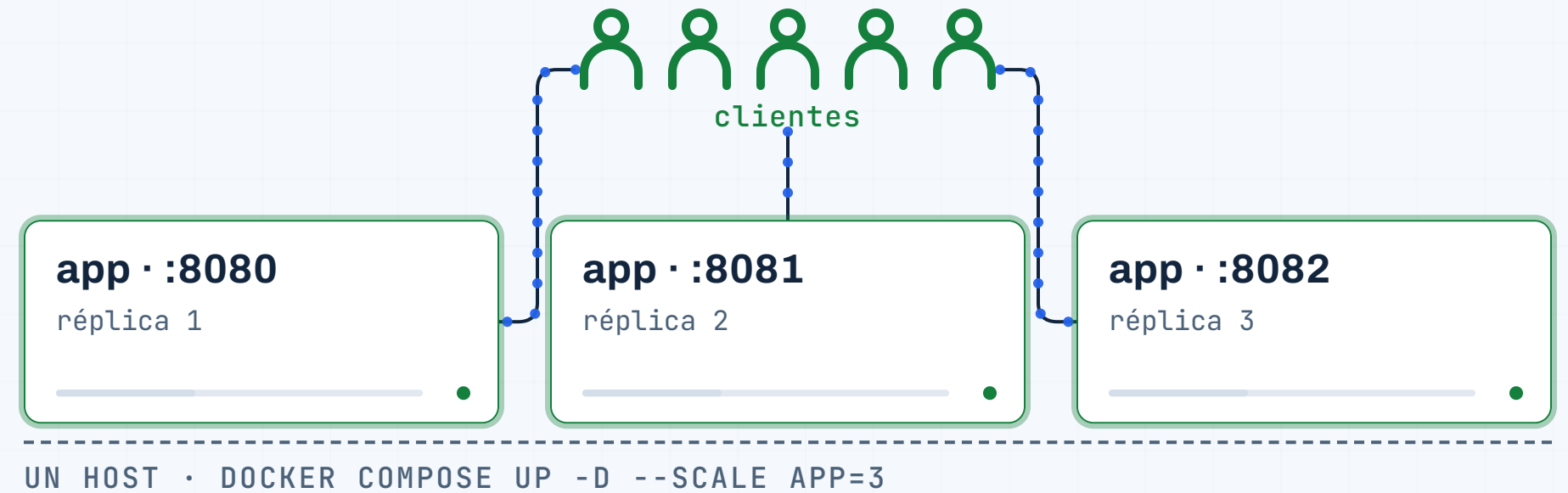
Un solo host.

LÍMITE DE COMPOSE

Compose soporta **despliegues de producción en hosts individuales**: su universo es una máquina.

CON `--SCALE APP=3`

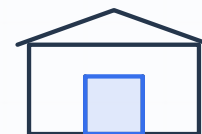
Tres puertos 8080, 8081, 8082, un host
Sin balanceo cada cliente elige puerto
Host caído caen las tres



SIN BALANCEADOR: NADIE REPARTE

DE UNO A MUCHOS

Pocos contenedores uno por uno, a mano
Un host un archivo y un comando
Muchas instancias guiones que ya no **escalan**



COMO SI FUERA

Compose es un administrador excelente de un solo edificio: **no puede mudar la empresa a otra ciudad.**

EN UNA FRASE Tres réplicas en una máquina **no son alta disponibilidad**: si se apaga el host, caen las tres.

Varias máquinas.

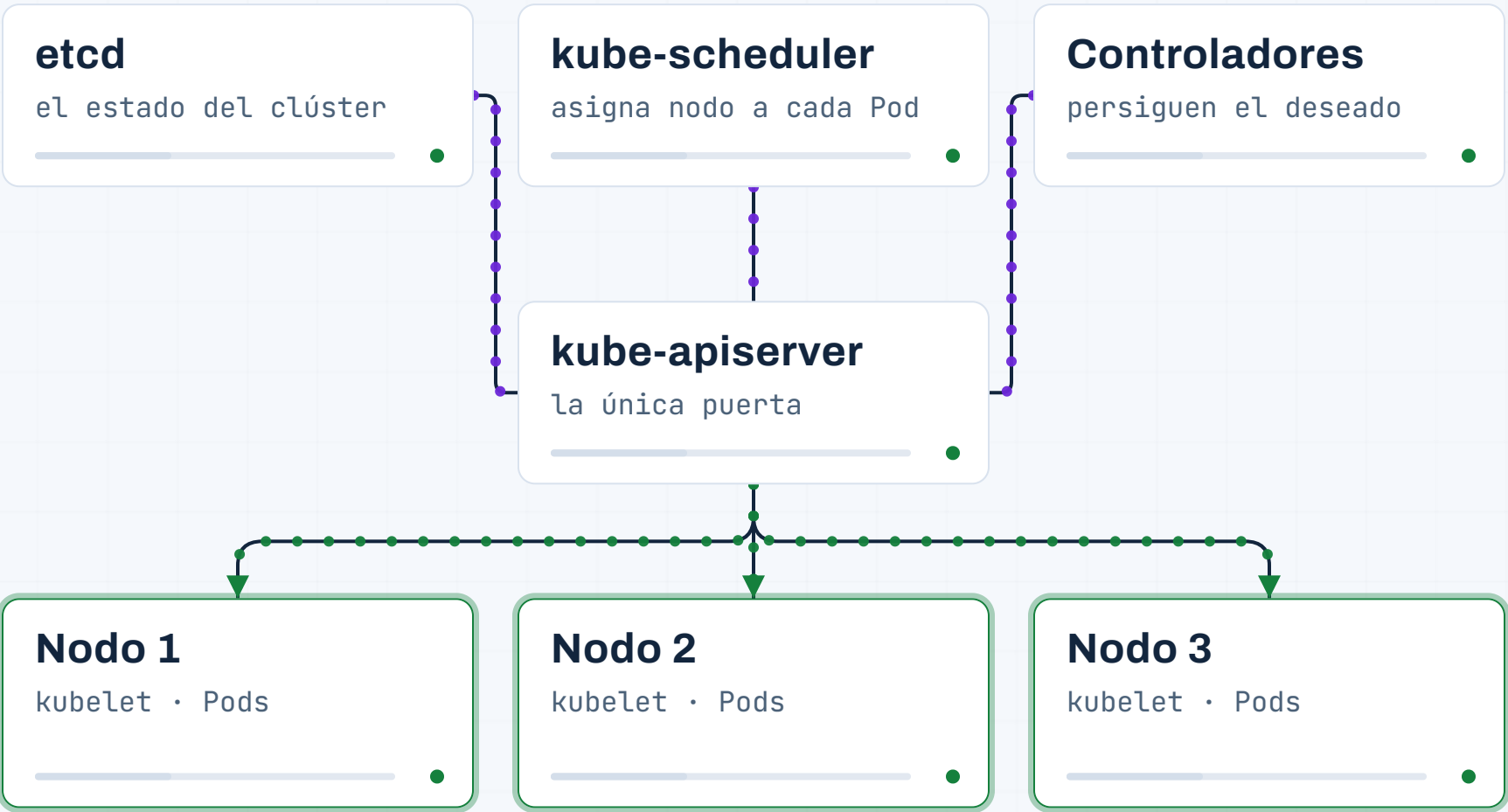
CLÚSTER

Un **plano de control** más un conjunto de nodos que corren las aplicaciones en contenedores.

QUIÉN HACE QUÉ

Nodos hospedan los Pods; kubelet en cada uno
Plano de control gobierna nodos y Pods
Producción el plano vive en **varias máquinas**

PLANO DE CONTROL



COMO SI FUERA

Una escuela con muchos salones y una sola dirección: **si un salón se inunda, la clase sigue en otro**, porque quien asigna salones no está adentro.

EN UNA FRASE

Un clúster no es una máquina más grande: **es un grupo de máquinas con alguien que decide por todas.**

En una frase.

KUBERNETES

Plataforma portátil, extensible y de código abierto para **gestionar cargas en contenedores**.

LA FRASE COMPLETA

Facilita configuración declarativa y automatización
Compose una máquina, hosts individuales
Kubernetes un clúster, **varias máquinas**

Docker Compose

una máquina

UN YAML, UN COMANDO

HOSTS INDIVIDUALES

UNA MÁQUINA

Kubernetes

un clúster

PORTÁTIL Y EXTENSIBLE

CÓDIGO ABIERTO

CONFIGURACIÓN DECLARATIVA

AUTOMATIZACIÓN

K8s = K + ubernete + s



COMO SI FUERA

La misma empresa que administraba un edificio, ahora con sucursales en varias ciudades: **hace falta quien decida por todas**.

EN UNA FRASE **Kubernetes no reemplaza a Docker: Docker corre contenedores y Kubernetes decide dónde y cuántos.**

Di qué quieres, no cómo.

ESTADO DESEADO

Describes lo que quieres y **Kubernetes cambia el estado real hasta alcanzarlo**, a un ritmo controlado.

CÓMO SE PIDE

Sí tres copias de **tienda:1.4** y una dirección estable
No arranca esto en esta máquina
kube-scheduler elige el nodo de cada Pod

Estado deseado

replicas: 3 · tienda:1.4

compara

CUENTA

deseado

3

real

2 → 3

ajusta

HAY 2, QUIERO 3: CREA 1

tienda-6b7a9-7f2a

nodo 1 · Running

tienda-6b7a9-9c4d

nodo 2 · Running

tienda-6b7a9-b81e

nodo 1 · nuevo



COMO SI FUERA

Le dices a la app del taxi **a dónde quieres llegar**, no por cuáles calles: si una se cierra, recalcula sola.

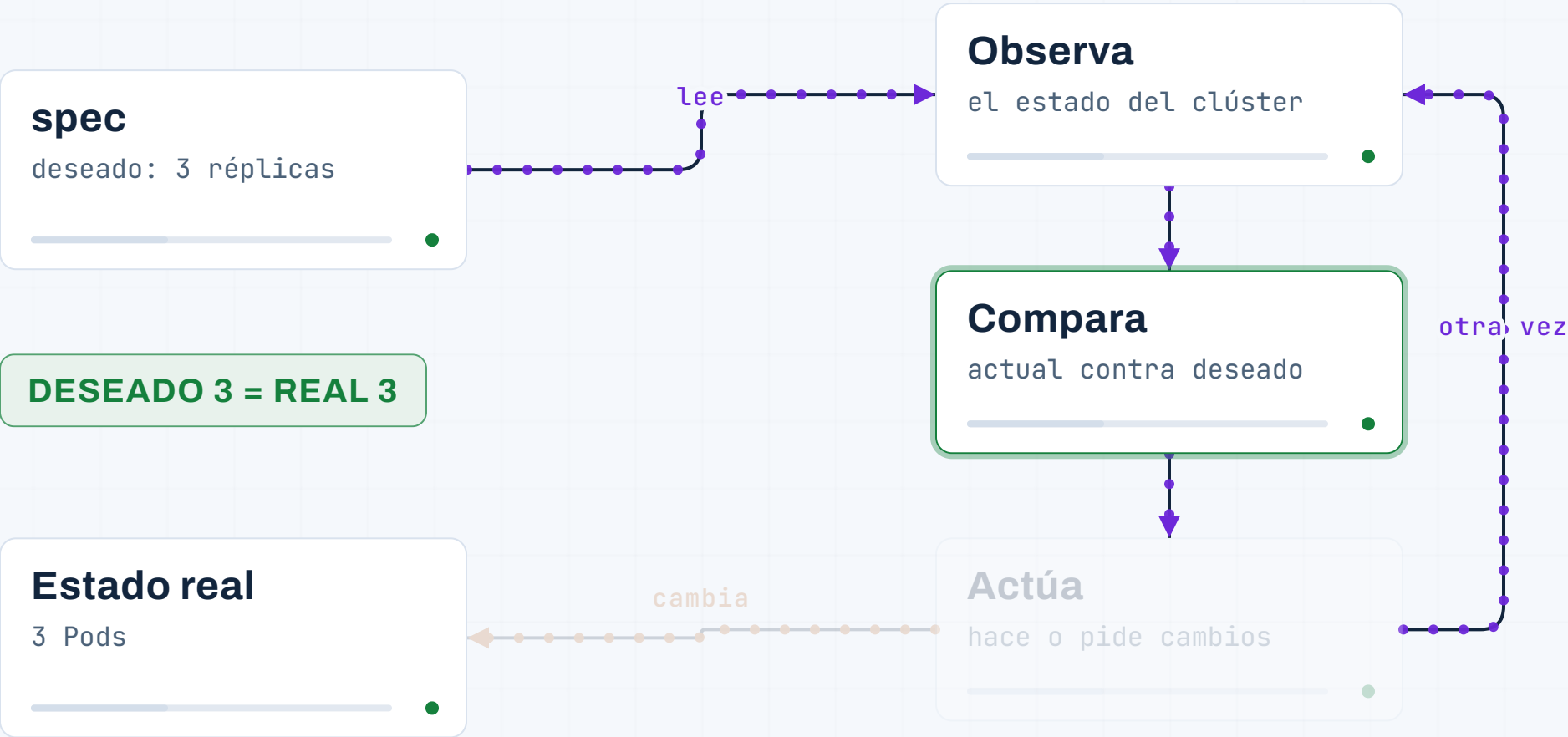
El termostato que nunca duerme.

CONTROLADOR

Bucle que observa el estado del clúster y **hace o pide cambios** para acercar el actual al deseado.

EL BUCLE

- Observa el estado del clúster
- Compara actual contra deseado
- Actúa hace o pide cambios, y **vuelve a empezar**



COMO SI FUERA
Pones la temperatura que quieres y te vas: **el termostato mide y corrige toda la noche**, sin que nadie se lo recuerde.

EN UNA FRASE Un controlador no se ejecuta y termina: **es un bucle que nunca deja de comparar.**

La unidad mínima.

POD

La unidad de cómputo más pequeña:
uno o más contenedores que comparten red y almacenamiento.

LO QUE HAY QUE SABER

Una IP por Pod; sus contenedores la comparten
Efímero desechable; rara vez se crea a mano
Lo crea un Deployment, no una persona

**COMO SI FUERA**

Dos escritorios en el mismo cubículo: **una sola extensión y un solo archivero** para los dos, y si se desarma el cubículo se van juntos.

Quien cuenta las copias.

DEPLOYMENT

Gestiona Pods sin estado; **debajo, el ReplicaSet mantiene estable el número de Pods** creando y borrando.

EN EL EJEMPLO

replicas: 3 el número deseado va en `.spec.replicas`
template la plantilla con la que se crea cada Pod
ReplicaSet crea y borra hasta llegar al número

Deployment

tienda · replicas: 3

ReplicaSet

tienda-6b7a9

REPLICASET DESEADO 3 · REAL 3

Pod

tienda-6b7a9-k2xq

Pod

tienda-6b7a9-m8rt

Pod

tienda-6b7a9-p7vd



COMO SI FUERA

El contrato del turno pide **tres personas en el piso**, no dice sus nombres: si una falta, entra otra y el número vuelve a tres.

EN UNA FRASE

Tú declaras cuántas copias quieres; **quien las cuenta y las repone es el ReplicaSet**, y nunca revive a la que se fue.

Nunca vuelve el mismo.

REEMPLAZO DE UN POD

Se programa una sola vez; **nunca se reprograma a otro nodo: se reemplaza por un Pod nuevo casi idéntico** con otro nombre y otra IP.

CUANDO MUERE EL NODO

Sin latido el kubelet deja de renovar su Lease y el nodo queda NotReady

El conteo el ReplicaSet ve dos y quiere tres

El nuevo otro nodo, otro nombre, otra IP

Nodo A

kubelet · Lease renovado

Pod

tienda-6b7a9-k2xq ·
10.244.1.7

CRASHLOOPBACKOFF **SUBE RESTARTS**

Nodo B

kubelet · Lease renovado

Pod

tienda-6b7a9-m8rt

Pod

tienda-6b7a9-p7vd



COMO SI FUERA

Si se cae el piso entero, nadie repara ese cubículo: **arman otro en otro piso, con otra extensión**, y el trabajo sigue en el nuevo.

Una puerta estable.

SERVICE

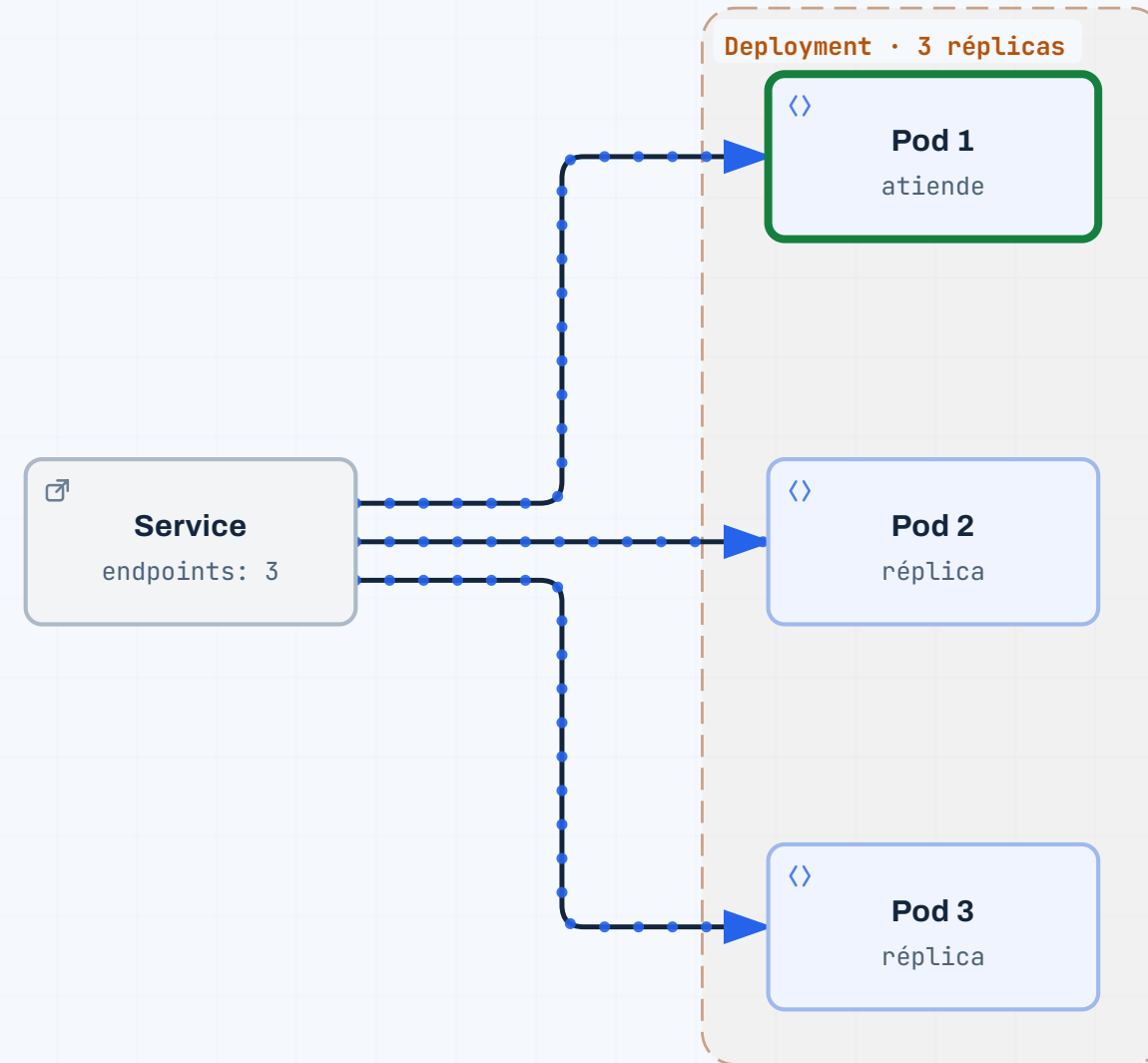
Expone una aplicación que corre en uno o más Pods: **los encuentra por etiquetas y mantiene la lista al día.**

LO QUE DA

Nombre DNS estable: tienda, aunque las IPs cambien

Endpoints la lista de Pods vivos, por etiqueta

kube-proxy en cada nodo, lleva el tráfico al Pod de esa lista



COMO SI FUERA

Una extensión que no cambia aunque **roten las personas que contestan**: kube-proxy es quien pasa cada llamada al escritorio que sí está ocupado.

EN UNA FRASE

Nadie se conecta a la IP de un Pod: **se conecta al nombre**, y kube-proxy pone la llamada en uno que esté vivo.

¿Qué predices?

COMPROBACIÓN 2

Un Deployment tienda con 3 réplicas; corres **kubectl delete pod** sobre uno de sus Pods. ¿Qué predices?

OPCIONES

- A** el mismo Pod, con el mismo nombre
- B** un Pod nuevo, otro nombre, AGE 0
- C** restaura el Pod de un respaldo

EN UNA FRASE

Comprométete con una opción antes de ver el resultado: **predecir primero es lo que hace que se quede.**

Pods

DEPLOYMENT · REPLICAS: 3

Service

tienda · 10.96.0.12

Pod

tienda-6b7a9-k2xq

¿?

tienda-6b7a9-m8rt

Pod

tienda-6b7a9-p7vd

kubectl delete pod tienda-6b7a9-m8rt

Tres copias, dos verdades.

FALLO PARCIAL

Tres copias de la verdad y la red se corta: **cada lado puede aceptar órdenes distintas**. Dos verdades.

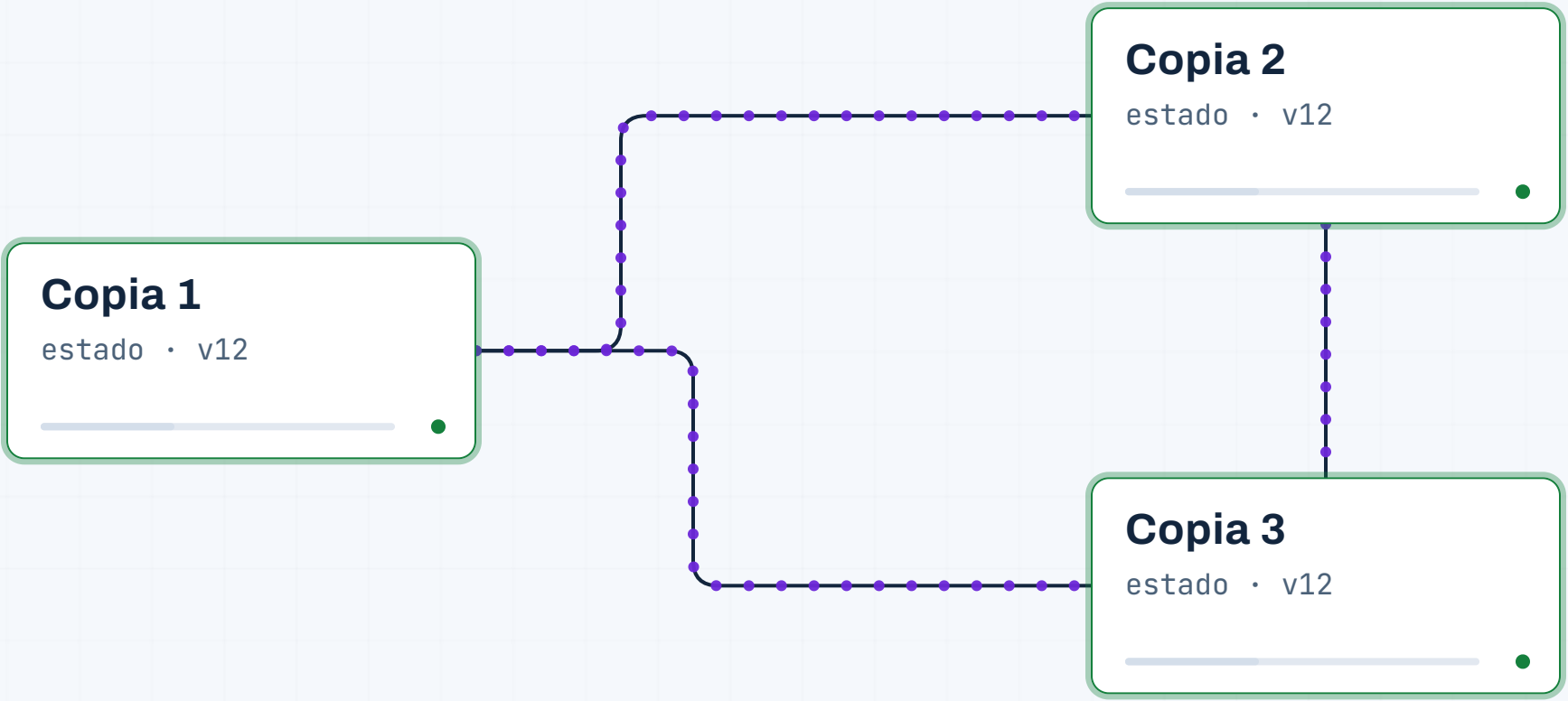
LO QUE HAY QUE IMPEDIR

Lado A acepta "crea un Pod"

Lado B acepta "borra el Deployment"

Producción el plano de control corre con **3 o 5 copias**

PLANO DE CONTROL • 3 COPIAS DEL ESTADO



COMO SI FUERA

Dos personas con copia de la misma lista de asistencia dejan de hablarse y cada una tacha un nombre distinto: **ya no hay una lista, hay dos.**

EN UNA FRASE

Copiar el dato no es ponerse de acuerdo: **sin quien decida, cada copia se queda con su verdad.**

Ponerse de acuerdo.

CONSENSO

Varios nodos tienen que **decidir el mismo valor** para una variable, y ese valor tiene que ser **uno de los que alguien propuso**.

LO QUE SÍ Y LO QUE NO

Deciden uno el mismo para todos los que deciden

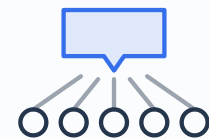
Sale de los propuestos no se inventa **ni se promedia**

Aunque fallen algunos el grupo sigue decidiendo



CINCO VALORES PROPUESTOS · NINGUNO DECIDIDO

EL PROMEDIO, 5.8, NO LO PROPUSO NADIE



COMO SI FUERA

El salón fija la hora del repaso: acordar no es promediar lo que dijo cada quien, es que **todos salgan repitiendo una hora que alguien propuso**.

EN UNA FRASE

Promediar no es decidir: el valor acordado tiene que salir de alguien, no de una cuenta.

Cuatro cosas que cumplir.

PROPIEDADES

Las **cuatro condiciones** que cualquier algoritmo de consenso tiene que cumplir para ser correcto.

LAS CUATRO

Terminación todo nodo que no falla **acaba decidiendo**

Acuerdo dos que deciden, deciden lo mismo

Validez lo decidido lo propuso alguien

Terminación

sin ella: el grupo se queda esperando para siempre

Integridad uniforme

cada nodo decide a lo más una vez

Acuerdo

dos que deciden, deciden lo mismo

Validez uniforme

el valor decidido lo propuso alguien

RÚBRICA · SE LE APLICA A PAXOS, A RAFT Y AL QUE VENGA

CUATRO CASILLAS · SE TACHAN O NO SE TACHAN



COMO SI FUERA

La rúbrica con la que calificas un proyecto: no preguntas si el algoritmo te cae bien, revisas las **cuatro casillas** y marcas la que no cumple.

EN UNA FRASE

Sirven para juzgar **cualquier** algoritmo de consenso, no solo el que vamos a ver.

El mismo problema, otros nombres.

CASOS PARTICULARES

Varios problemas que parecen distintos son **el mismo problema de consenso** con otro nombre y otra variable.

LA VARIABLE QUE SE ACUERDA

Elección de líder quién manda

Pertenencia al grupo quiénes siguen vivos

Compromiso distribuido la transacción va o **se deshace**

Elección de líder

la variable es quién manda

Pertenencia al grupo

la variable es quiénes siguen vivos

Compromiso distribuido

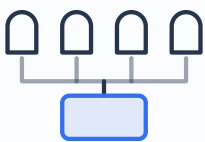
la variable es si la transacción va

Difusión ordenada

la variable es en qué orden llegan

Consenso

decidir un valor entre varios



COMO SI FUERA

Cuatro ventanillas con cuatro letreros distintos y, al llegar, **el mismo formato en las cuatro**: cambia el trámite, no el procedimiento.

Un algoritmo con nombre.

RAFT

Un algoritmo de consenso diseñado para ser **entendible**, con las mismas garantías que el que ya existía.

DE DÓNDE VIENE

Paxos, 1998 correcto y difícil de implementar bien

Raft, 2013 Ongaro y Ousterhout, Stanford

Supone caídas nodos que se caen y vuelven, **no que mienten**

Paxos

Lamport, 1998 · correcto y difícil de implementar bien

mismas garantías

Raft

Ongaro y Ousterhout, Stanford, 2013

Lo que Raft tolera

caídas, red partida, mensajes tarde o repetidos

Lo que Raft no cubre

nodos que mienten: fallos bizantinos, otra liga

**COMO SI FUERA**

Decir antes de empezar qué se vale que falle es **acordar las reglas antes del partido**: sin reglas escritas, cualquier jugada se discute al final.

Un latido, y si falta, elección.

HEARTBEAT

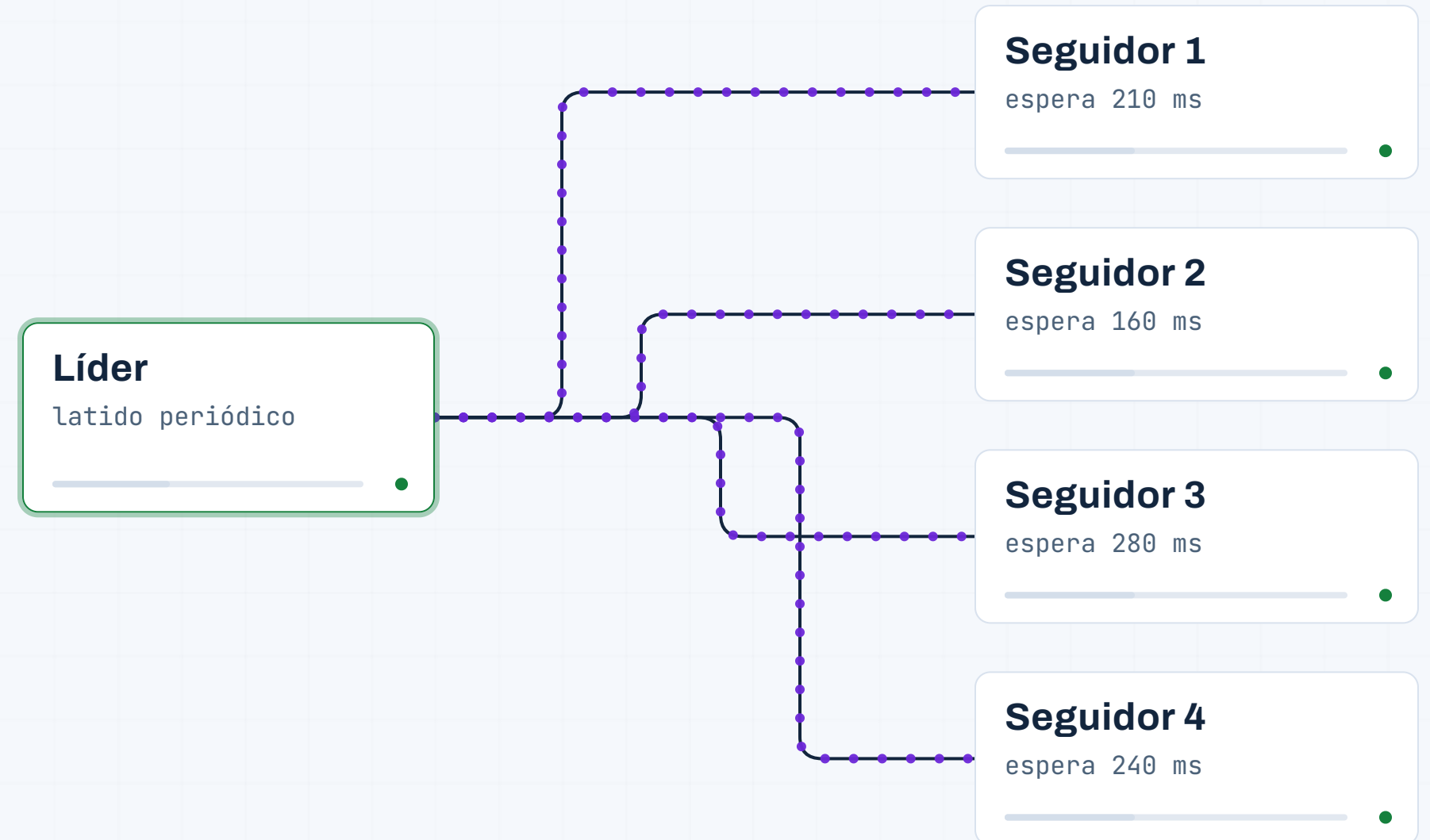
El líder manda latidos periódicos; quien no los oye en su tiempo de espera **se postula y pide votos**.

CÓMO SE ELIGE

Espera al azar 150 a 300 ms, **distinta en cada servidor**

Un **voto** por servidor y por término, que se define enseguida

Votos divididos nadie gana; otra ronda con espera nueva



COMO SI FUERA

Un grupo de mensajes cuyo administrador lleva rato callado: **el primero que se cansa de esperar pide el puesto**, y solo lo obtiene si más de la mitad responde que sí.

EN UNA FRASE

Nadie de afuera repone al líder: **el propio grupo lo repone votando**.

Un término

por líder.

TÉRMINO

Un **periodo con un solo líder**: cada elección abre un término nuevo y el número nunca retrocede.

PARA QUÉ SIRVE

Un **voto** cada servidor vota **una vez por término**

Nunca retrocede el número solo sube

Ordena el tiempo lo que llega de un término viejo se descarta

LÍNEA DE TIEMPO DEL CLÚSTER

Término 1

líder A

Elección

sin líder

Término 2

líder B

el número solo sube

NÚMERO DE TÉRMINO

en curso

1

manda

líder A



COMO SI FUERA

Los periodos de una mesa directiva: **se numeran los periodos, no a las personas**, y quien llega con un acuerdo del periodo anterior se queda sin efecto.

El log

es la verdad.

LOG REPLICADO

Cada nodo guarda la **lista ordenada de operaciones**, no el resultado; repetir la lista desde el principio reproduce el estado.

POR QUÉ ASÍ

Se replica el **qué** la **operación**, no el resultado

Mismo orden todos aplican lo mismo en la misma secuencia

Cada entrada lleva su índice y su término

TRES NODOS · LA MISMA LISTA, EN EL MISMO ORDEN

NODO 1 · LOG

1 · t1 x = 3

2 · t1 y = 7

3 · t2 x + 5

NODO 2 · LOG

1 · t1 x = 3

2 · t1 y = 7

3 · t2 x + 5

NODO 3 · LOG

1 · t1 x = 3

2 · t1 y = 7

3 · t2 x + 5

ESTADO · X = 8 · Y = 7

ESTADO · X = 8 · Y = 7

ESTADO · X = 8 · Y = 7

MISMAS OPERACIONES Y MISMO ORDEN · MISMO ESTADO



COMO SI FUERA

La libreta de movimientos de una tienda: no se copia el saldo final, se copian **los movimientos en orden**, y con esa lista cualquiera vuelve a sacar el saldo.

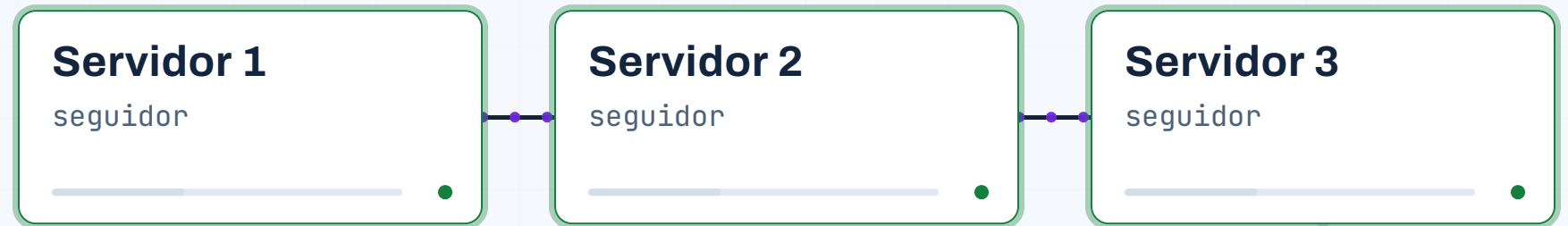
La mayoría del clúster completo.

MAYORÍA

Más de la mitad de **los servidores del clúster completo**, no de los que siguen vivos.

CON LA DEFINICIÓN EXACTA

5 servidores toleran el fallo de 2 cualesquiera
Se cuenta sobre los 5, **no sobre los vivos**
2 vivos de 5 no deciden nada



CLÚSTER DE 5

mayoría

tolera

**3 de 5
2 caídos**

MAYORÍA A · 1, 2, 3 · 3 DE 5

Servidor 4
seguidor

Servidor 5
seguidor



COMO SI FUERA

El quórum de una asamblea: se cuenta sobre **toda la lista de socios**, no sobre los que llegaron, y tres ausentes no vuelven mayoría a los dos presentes.

EN UNA FRASE

Dos mayorías del mismo clúster siempre comparten un servidor: por eso no puede haber dos verdades.

¿Cuándo está

comprometida?

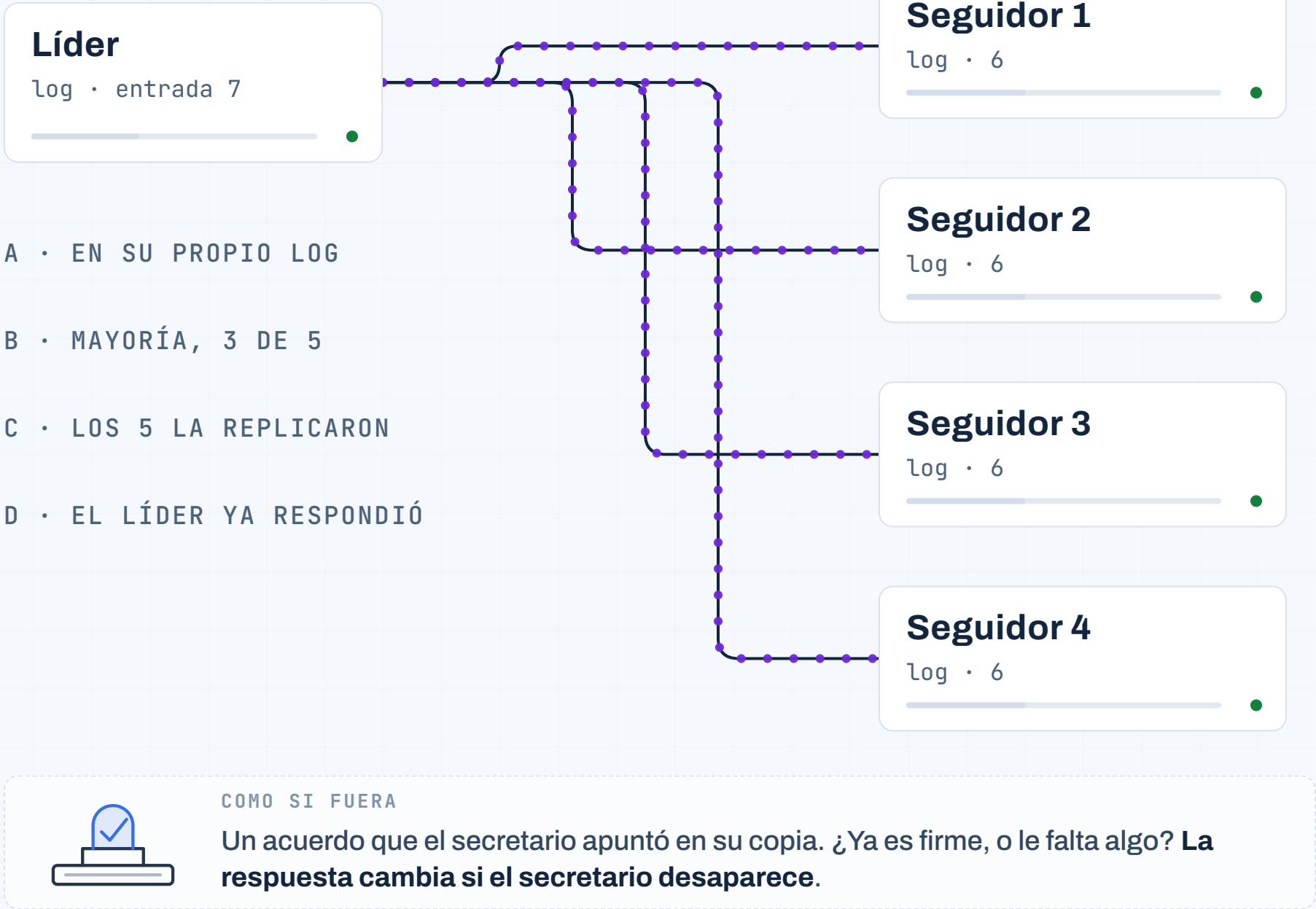
PREGUNTA DE LA SEMANA

Raft, 5 servidores. El líder manda una entrada al log. ¿Cuándo la considera **comprometida**?

ANTES DE RESPONDER

Escrita el líder ya la puso en su propio log
Comprometida ya **no se puede perder**
Elige una A, B, C o D, y di por qué

EN UNA FRASE No preguntes si está escrita: pregunta **qué sobrevive si el líder desaparece.**



El que se quedó atrás.

REPARACIÓN DEL LOG

El líder **retrocede hasta la última entrada en la que coinciden** y de ahí para adelante le reenvía todo lo que le falta.

CÓMO LO REPARA

Compara índice y término de la última entrada

Retrocede hasta el punto donde **los dos coinciden**

Reenvía de ahí en adelante, en orden

CADA ENTRADA · ÍNDICE · TÉRMINO · OPERACIÓN

LÍDER · LOG COMPLETO

1 · t1 x = 3

2 · t1 y = 7

3 · t2 x + 5

4 · t2 x - 2

REZAGADO · VUELVE

1 · t1 x = 3

2 · t1 y = 7

falta 3 estuvo caído

falta 4 estuvo caído



COMO SI FUERA

El compañero que faltó no vuelve a copiar el cuaderno entero: busca **la última clase que sí tiene** y copia de ahí para adelante.

EN UNA FRASE El log del líder manda: **el que volvió se alinea, no negocia.**

Cuánto dura sin líder.

REGLA DE TIEMPOS

Difusión mucho menor que la espera de elección, y esa mucho menor que el tiempo entre fallos: **0.5 a 20 ms**, **10 a 500 ms**, **meses**.

MEDIDO EN EL PAPER

Peor caso **513 ms** con 50 ms de azar

Sin azar más de 10 s por votos divididos

Espera muy corta cambios de líder innecesarios

DIFUSIÓN · 0.5 A 20 MS

ELECCIÓN · 10 A 500 MS

ENTRE FALLOS · MESES

ESCALA LOGARÍTMICA · X10 POR TRAMO

TIEMPO SIN LÍDER

peor caso

513 ms

azar

50 ms

ESPERAS DISTINTAS · UNO GANA



COMO SI FUERA

Cinco personas esperando a alguien que no llega: si todas aguantaran **exactamente lo mismo**, saldrían a buscarlo al mismo tiempo y se estorbarían en la puerta.

EN UNA FRASE

Acortar la espera no siempre ayuda: por debajo de cierto punto se cambia de líder sin motivo.

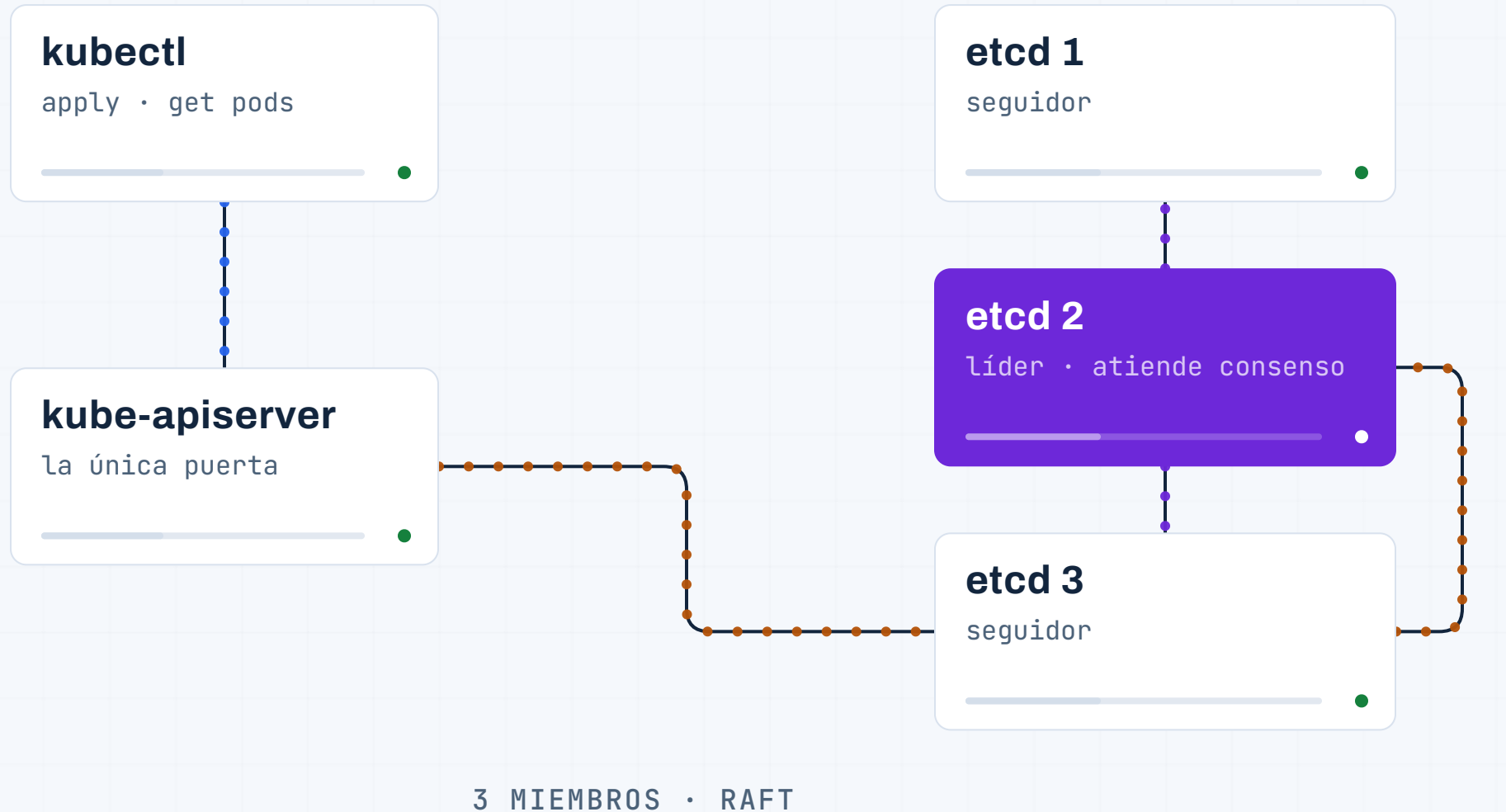
Dónde vive el acuerdo.

ETCD

Almacén llave-valor consistente y de alta disponibilidad, **respaldo de todos los datos del clúster.**

DENTRO DE KUBERNETES

kube-apiserver la única puerta hacia etcd
Raft por líder un seguidor **reenvía al líder**
No está en etcd no existe en el clúster



COMO SI FUERA

La ventanilla contesta y sonríe, pero **lo que vale es el libro de atrás**: si ahí no quedó escrito, para la escuela nunca pasó.

EN UNA FRASE

El API server no guarda nada: es la puerta, y la memoria del clúster está detrás de ella.

Impares,

y no más de siete.

QUÓRUM

$(n/2)+1$. Un clúster impar **tolera los mismos fallos** que uno par, con menos nodos; no más de siete.

FALLOS QUE TOLERA

3 miembros mayoría 2, tolera **1**

4 miembros mayoría 3, tolera **1**

5 miembros mayoría 3, tolera **2**

n = 3

mayoría 2 · tolera 1



QUÓRUM $(N/2)+1$

mayoría

2 de 3

tolera

1 fallo

n = 4

mayoría 3 · tolera 1

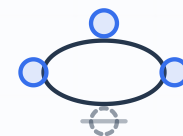


n = 5

mayoría 3 · tolera 2



TECHO: 7 MIEMBROS



COMO SI FUERA

Una mesa de cuatro necesita tres votos para decidir, los mismos que una de tres: **la cuarta silla no compra nada** y hay una ausencia más que cubrir.

EN UNA FRASE

La tolerancia solo sube de impar a impar: **el miembro par paga sueldo y no vota de más.**

Réplicas o máquina más grande.

ESCALAR

Cambiar el **número de réplicas** (horizontal) o los **recursos de cada una** (vertical).

QUÉ EXIGE CADA UNA

Horizontal procesos **sin estado** y algo que reparta
Vertical un solo punto de falla; no cambia código
etcd cinco con **discos rápidos**, no diez lentos



tráfico x 3

tienda

app · sin estado

A · HORIZONTAL

réplica 1

mismo tamaño

réplica 2

mismo tamaño

réplica 3

mismo tamaño

B · VERTICAL

1 instancia

más CPU y RAM



COMO SI FUERA

Un tráiler de treinta toneladas se detiene con **una llanta pinchada**; treinta camionetas siguen andando si una choca, mientras ningún chofer se sepa la ruta de memoria.

La decisión

de hoy.

LA DECISIÓN

La aplicación del ejemplo recibe **el triple de tráfico**: ¿réplicas o máquina más grande?

EN TRES LÍNEAS

Elección una de las dos

Porque la razón, con el requisito enfrente

Sacrificio lo que pierdo al elegir

tienda

app web · tráfico x 3

ELECCIÓN

PORQUE

SACRIFICIO

Réplicas

horizontal · n copias
iguales

Máquina más grande

vertical · 1 instancia



COMO SI FUERA

Firmar sin leer la letra chica: **la decisión se cumple igual**, y el costo que no dijiste aparece solo, más tarde y más caro.

EN UNA FRASE

Aquí no hay respuesta correcta: la respuesta completa es la elección, la razón y el sacrificio.

Descanso.

Mexi Banco arriba.

DESCANSO

Al regresar, Mexi Banco corriendo en su máquina: **clone** y **compose up**.

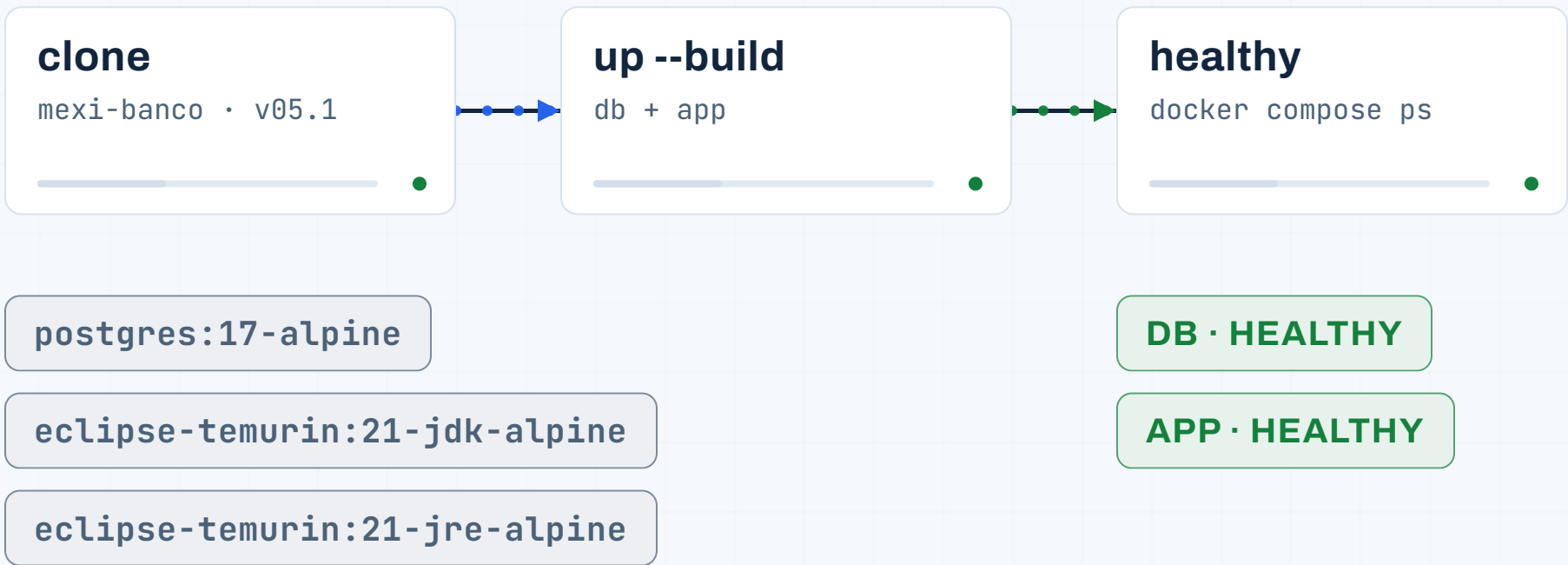
MIENTRAS TANTO

git clone --branch v05.1
github.com/OscarRuiz21/mexi-banco

docker compose up --build la primera vez
descarga tres imágenes

Si algo truena lo vemos al regresar, **no lo peleen solos**

MEXI-BANCO · V05.1 · DB + APP



EN UNA FRASE

Hasta aquí la teoría: lo que sigue ya es código que corre en tu máquina.

Un banco, cinco piezas.

UN MONOLITO

Cinco piezas en **un solo deployable**:
un proceso Java y una base de
datos.

CUATRO DE LAS CINCO

- Cuenta CLABE y saldo
- Movimiento asiento inmutable: el ledger solo crece
- Transferencia interna, una sola transacción
- SPEI externo, lento e irreversible

MEXI-BANCO V05.1 · UN SOLO DEPLOYABLE · UNA SOLA BASE

Cuenta

CLABE, saldo

Movimiento

asiento inmutable

Transferencia

interna · 1 transacción

SPEI

externo, lento,
irreversible

Notificación

stub · escribe en el
log



COMO SI FUERA

Todo el banco en un solo cubículo: cinco escritorios que se hablan de viva voz, **sin salir nunca al pasillo.**

Qué se puede pedir.

CUATRO ENDPOINTS

Abrir, consultar, transferir, SPEI. Y /actuator/health: lo que healthcheck y probes **preguntan**.

REST, COMO EN LA T01

- POST /cuentas
- GET /cuentas/{clabe}
- POST /transferencias
- POST /spei + Idempotency-Key



curl

POST /cuentas · 201

app

:8080 ·
/actuator/health

db

INSERT cuenta



COMO SI FUERA

La carta del restaurante: cuatro platillos que se pueden pedir y, en la puerta, **el letrero** que dice si la cocina está abierta.

EN UNA FRASE Un servicio se conoce por lo que se le puede pedir: **cuatro peticiones, y una quinta que solo pregunta si está listo.**

Un sistema

que crece.

UNA PIEZA POR SESIÓN

Hoy monolito. Cada sesión agrega **la pieza que ese día se explica:** partido, resiliente, eventos, saga.

LO QUE VIENE

- v06 se parte en tres; gateway y discovery
- v08 resiliencia y trazas
- v10 eventos y notificaciones reales
- v12 la saga del SPEI



COMO SI FUERA

La línea de ensamblaje: el banco recorre una estación por sesión y **sale de cada una con una pieza más de la que entró.**

EN UNA FRASE

De la S06 en adelante este monolito se va partiendo en servicios, y se hace **leyendo el código en clase, no de memoria.**

Tres hitos,

con tus manos.

CON TUS MANOS

Arriba y respondiendo, las propiedades de Compose, la idempotencia otra vez. **Cada hito termina con algo que se ve.**

ENTREGA

docker compose ps ambos servicios healthy
Segundo SPEI misma clave, sin doble cobro
README las tres preguntas de cierre

Hito 1 · Arriba y respondiendo

`compose up --build` · abrir y consultar una cuenta

Hito 2 · Propiedades de Compose

`network inspect` · transferencia · `stop`
app: no vuelve · `start app`

Hito 3 · Idempotencia otra vez

mismo SPEI, misma clave, un solo cobro

Para la S6.

PARA LA S6

Consistencia y CAP, ya con Raft
dado. Mexi Banco se parte en tres y
la transferencia **deja de funcionar**.

LO QUE TRAES

Lectura 5 Brewer, CAP Twelve Years Later
(2012)

Mexi Banco corriendo en Compose, como hoy



49 / CIERRE

Ocho preguntas

que ya puedes responder.

OCHO PREGUNTAS

Si puedes contestarlas sin notas, **la sesión quedó**. Si una no, esa es la que repasas con el estudio.

COMPOSE

- 1 ¿Cómo encuentra la aplicación a la base de datos si nadie le dijo su IP?
- 2 ¿Qué pasa si la aplicación arranca antes que la base de datos?
- 3 ¿Quién vuelve a levantar un contenedor que se cayó solo?
- 4 ¿Hasta dónde llega Compose y por qué una sola máquina es el techo?

KUBERNETES, CONSENSO Y RAFT

- 5 ¿Qué significa declarar el estado deseado en vez de dar órdenes?
- 6 ¿Qué pasa exactamente, paso a paso, cuando muere un Pod?
- 7 ¿Por qué elegir líder es solo un caso particular de consenso, y qué más lo es?
- 8 ¿Qué se replica en Raft, el estado o las operaciones, y cuándo queda comprometida una entrada?

EN UNA FRASE

Ninguna de las ocho se contesta con una definición: **todas piden que expliques el mecanismo.**

Fuentes

de la sesión.

FUENTES

Todo lo de hoy sale de fuentes **primarias**: la documentación oficial de Docker, Kubernetes y etcd, el paper de Raft y 12factor.net.

DOCUMENTACIÓN

docs.docker.com contenedor, Compose, networking, healthcheck, volúmenes, restart

kubernetes.io/docs architecture, controller, Pods, Deployment, ReplicaSet, Service, etcd

etcd.io/docs/v3.6 faq, tuning, hardware

12factor.net factor III, configuración; factor VI, procesos

PAPERS

Ongaro y Ousterhout, 2014 In Search of an Understandable Consensus Algorithm (Raft); raft.github.io, con el simulador

Brewer, 2012 CAP Twelve Years Later: How the Rules Have Changed; Lectura 5

EN UNA FRASE

Si algo de esta sesión no coincide con la fuente, **gana la fuente.**

Ya lo comprobaste.

EL PRINCIPIO

Compose asume que la máquina no falla; **Kubernetes** asume que sí, y automatiza la reacción.

ANTES DE CERRAR

Ticket de salida ¿acertaste tu predicción? ¿qué fue lo más confuso?

Lectura Raft §5: vuelve sobre lo que ya viste

Próxima S06 · consistencia y CAP; Mexi Banco en tres

